View Planned Event Details

Context

To view the details of a planned event, do the following:

Procedure

1. Launch the CPE app.
2. The overview page displays the list of process types for all the active models.
3. Select a process type from the list and click it.
4. The **Process List** page displays a table of all active processes.
5. Select a process from the list and click it.
6. The **Process Details** page shows the information of the selected process arranged by:
    - **Information**
    - **Planned Events**
    - **Other Events**
7. Click **Planned Events** to see the details

Related Information

Planned Event Details

View Actual Event Details

Context

To view the details of an actual event, do the following:

Procedure

1. Launch the CPE app.
2. The overview page displays the list of process types for all the active models.
3. Select a process type from the list and click it.
4. The **Process List** page displays a table of all active processes.
5. Select a process from the list and click it.
6. The **Process Details** page shows the information of the selected process arranged by:
    - **Information**
    - **Planned Events**
    - **Other Events**
7. Click **Other Events** to see the details of actual events without reference to any planned event.
8. To check the actual data with reference to the planned event, on the **Planned Events** tab, click a planned event in the **Planned Events** table to display the **Planned Event Details** page. Then on the **Actual Data** tab, you can drill down to view

details.

Related Information

Event Details

Appendix: Event to Action Script

An event-to-action script allows you to incorporate logic (business rules) into your model. This section explains about the structure and syntax of the script.

To add your script to a model, see the section Add a Script to Your Model.

To know more about the notification service, see the guide How to Send Notifications.

Lexical Structure

The lexical structure of a programming language is the set of elementary rules that specifies how you write programs in that language. It is the lowest-level syntax of a language. It specifies such things as what variable names look like, the delimiter characters for comments, and how one program statement is separated from the next. This section documents the lexical structure of the event-to-action script.

Character Set

Case Sensitivity

This script is a case-sensitive language. This means that language keywords, variables, function names, and other ***identifiers*** must always be typed with a consistent capitalization of letters. The `while` keyword, for example, must be typed “while,” not “While” or “WHILE.” Similarly, online, Online, OnLine, and ONLINE are four distinct variable names.

Whitespace

This script recognizes the following characters as whitespace:

- space(\u0020)
- tab(\t)
- carriage return: (\r)
- line feed (\n)

Comments

This script supports the following styles of comments:

- Any text between a double slash "//" and the end of a line is treated as a comment and is ignored.
- Any text between the characters "/*" and "*/" is also treated as a comment. These comments may span multiple lines but may not be nested.

Literals

A literal is a data value that appears directly in a program. The following are all literals:

```
12                          // The Integer number
1.2                         // The float number
"hello world"              // A string value
true                       // A Boolean value
false                      // The other Boolean value
null                       // A null value
[1,2,3,4,5]               // An array
```

Identifiers and Reserved Words

Identifiers

An ***identifier*** is simply a name. Identifiers are used to name variables and functions and to provide labels for certain loops in the script code. An identifier must begin with a letter, or an underscore "_". Subsequent characters can be letters, digits, or underscores. The following are all legal identifiers:

```
a
a_b_c
x11
_test
```

Reserved Words

The script reserves a number of identifiers as the keywords of the language itself. You cannot use these words as identifiers in your programs:

- actualEvent
- plannedEvent
- trackedProcess
- break
- function
- return
- case
- do
- if
- switch
- var
- else
- continue

- false
- while
- finally
- true
- for

Semicolons

Like many programming languages, we use the semicolon ";" to separate statements from each other. This is important to make the meaning of your code clear. Without a separator, the end of one statement might appear to be the beginning of the next, or vice versa.

Values and Variables

String Literals

To include a string literal in a program, simply enclose the characters of the string within a matched pair of double quotes ("). The maximum size for a string is 10M. Here are examples of string literals:

```
"testing"
"π is the ratio of a circle's circumference to its diameter"
```

There are three functions for a string literal:

1. Function `length()` returns the length of a specified string. For example:

   ```
   var str = “circle”;
   str.length(); // returns 6
   ```

2. Function `search()` searches for a specified string value and returns the position of the match. If no matches are found, the function returns -1. For example:

   ```
   var str = “π is the ratio of a circle's circumference to its diameter”;
   str.search("is"); // returns 2
   str.search("t"); // returns 5
   ```

3. Function `substring()` extracts the characters in a string from the start character to the end character and returns a new substring. To use this function, the following points should be noticed:

   - The start index is mandatory. If the end index is not specified, the function extracts from the start character to the end of the string.
   - The start character is included in the substring, but the end one is not.
   - Only integers are input as indices. Subtractive integers are recognized as 0.
   - The index 0 suggests the first character.

- If the start index is greater than the end index, such as `str.substring(4, 1)`, the indices will automatically swap places to `str.substring(1, 4)`.

Here are examples:

```
var str = “π is the ratio of a circle's circumference to its diameter”;
str.substring(2, 4); // returns “is”
str.substring(5); // returns “the ratio of a circle's circumference to its diameter”
```

Boolean Values

A boolean value represents true or false, on or off, yes or no. There are only two possible values of this type. The reserved words ***true*** and ***false*** evaluate to these two values.

null Value

null is a language keyword that evaluates to a special value that is usually used to indicate the absence of a value.

Object Value

An object is a composite value. It aggregates multiple values (primitive values or other objects) and allows you to store and retrieve those values by name.

An object is an unordered collection of properties, each of which has a name and a value. Property names are strings, so we can say that objects map strings to values. This string-to-value mapping goes by various names. The maximum size for an object is 100M.

Function `containsProperty()` checks if an object contains a property. The syntax of this function is:

```
object.containsProperty(“propertyName”)
```

For example:

```
o.containsProperty(“x”)
```

Array Value

An array is an ordered collection of values. Each value is called an element, and each element has a numeric position in the array, known as its index. An array element may be of any type, and different elements of the same array may be of different types. Array elements may even be objects or other arrays, which allows you to create complex data structures, such as arrays of objects and arrays of arrays.

There are two functions for an array value:

1. Function `array.size()` returns the size of the array.
2. Function `array.push(x)` adds the element x to the array as the last element.

Numbers

A number value can be an integer of float. A number value can be transformed to a string in this way:

```
num.toString()
```

Timestamp Value

There are four functions for a timestamp value:

1. Function `timestamp.minusDays(x)` subtracts the number of days (x) in the timestamp.
2. Function `timestamp.plusDays(x)` adds the number of days (x) in the timestamp.
3. Function `timestamp.minusMinutes(x)` subtracts the number of minutes (x) in the timestamp.
4. Function `timestamp.plusMinutes(x)` adds the number of minutes (x) in the timestamp.

Expressions and Operators

Property Access Expressions

A property access expression evaluates to the value of an object property or an array element.

1. Syntax for an Object value property access:

   ```
   expression . identifier
   ```

   Here are some examples:

   ```
   var o = toObject("{x:1,y:{z:3}}");                    //An example object
           o.x;                                           //=>1
           o.y.z;                                        //=>3
   ```

2. Syntax for Array value property access:

   ```
   expression [ expression ]
   ```

   Here are some examples:

   ```
   var o = toObject(" {x:1,y:{z:3}}");                   //An example object
           var a = [o,4,[5,6]];                                    // An example array that c
           a[0].x;                                                 //=>1
   ```

Invocation Expressions

An invocation expression is the syntax for calling (or executing) a function or method. It starts with a function expression that identifies the function to be called. The function expression is followed by an open parenthesis, a comma-separated list of zero or

more argument expressions, and a close parenthesis.

Here are some examples:

```
toObject(“( )”);            //toObject is the function expression; “( )” is the argument expressior
var uuid = toUUID("00000000-0000-0000-0000-000000000000");
var date = toDate("2010-05-04");
var timestamp = toTimestamp("2013-06-25T16:22:52.966Z");
```

Operators

Operators are used for arithmetic expressions, comparison expressions, logical expressions, assignment expressions, and more.

Here are all allowed operators:

Operator	Operation	Example
+	Positive number	+1
-	Negate number	-1
+,-	Add, subtract	1+2 1-2
<,<=,>,>=	Compare	a>=b
==	Test for equality	a==b
!=	Test for inequality	a!=b
^	Power	2^3 == 8
&&	Compute logical AND	true && false
||	Compute logical OR	true || false
!	Invert boolean value	!true
=	assignment	a=1

Statements

Declaration Statements

The “var” statement declares a variable. Here is the syntax:

```
var name [ = value]
```

Return Statement

The return statement terminates the execution of the code block immediately.

Conditions

Conditional statements execute or skip other statements depending on the value of a specified expression. These statements are the decision points of your code, and they are also sometimes known as “branches.”

The “if” statement is the fundamental control statement that allows script to make decisions, or, more precisely, to execute statements conditionally. This statement has 3 forms:

1. The first form’s syntax is:

```
if (expression)
           statement
```

2. The second form of the “if” statement introduces an else clause that is executed when the expression is false. Its syntax is:

```
if (expression)
    statement1
else
    statement2
```

3. The third form’s syntax is:

```
if (expression)
    statement1
else if(expression2)
    statement2
else
   statement2
```

Loops (while)

The script supports “while” statement for loop. It has the following syntax:

```
while (expression)
statement
```

To execute a “while” statement, the interpreter first evaluates the expression:

- If the value of the expression is false, then the interpreter skips over the statement that serves as the loop body and moves on to the next statement in the program.
- If the expression is true, the interpreter executes the statement and repeats, jumping back to the top of the loop and evaluating the expression again.
- The ***continue*** statement terminates the execution of the statements in the current iteration of the current loop and continues the execution of the loop with the next iteration.
- The ***break***statement terminates the current loop statement and transfers the program control to the statement following the terminated statement.

For example:

```
var i = 0;
var n = 0;

while (i < 5) {
  i = i+1;
  if (i == 3) {
    continue;
  }
  n = n+i;
  if( n > 8) {
   break;
}
          }
```

Event to Action Log Statement

The Event to Action Log statement begins a code line with “?” and is followed by a variable.

For example:

```
var test = "gtttest5";
?test ;
```

To see your event-to-action logs, you should do the following in the MM app first:

1. Deploy the model
2. Enable the Event to Action Log of the model.

After the message processing, the logs are displayed on the **Correlation Details** page of the Manage Message Logs app and the Manage Event Logs app. For instructions, see View Correlation Payload and Logs.

Related Information

Deploy a Model
Enable/Disable Event to Action Log of the Model

User-Defined Function

You can define your own functions. It must follow the syntax below:

```
function functionName(param1, param2, param3…..) {
                statement
}
```

For example:

```
function sample(a, b){
?a;
?b;
return a+b;
}
var c=sample(1,3); // c is 4 after execution
```

Note

You cannot use system embedded functions in your user-defined functions.

Samples

Notification service can be enabled by the event-to-action script. For details, see the section Implement the Script in the guide *How to Send Notifications*.

System Embedded Constants

There are some predefined system constants that you can invoke directly in the script. They include the following:

`trackedProcess`

The constant `trackedProcess` is an object of the tracked process. For its object structure example, see Example of trackedProcess Object.

Function `getFinalPlannedEvents()` under the `trackedProcess` object returns a list of events that are flagged as final event. For example:

```
trackedProcess.getFinalPlannedEvents();
```

`actualEvent`

The constant `actualEvent` is an object of the actual event received. For its object structure example, see Example of actualEvent Object.

`plannedEvent`

The constant `plannedEvent` is an object of the planned event. For its object structure example, see Example of plannedEvent Object.

Function `isReported()` under `plannedEvent` object returns a Boolean value: true or false. For example:

```
plannedEvent.isReported();
```

`modelNamespace`

The constant `modelNamespace` is a string value of the current model namespace. For example: